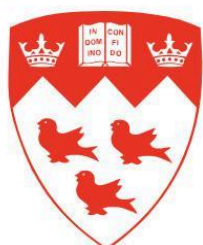


ECSE 458 Capstone Design Project

DP 06 – Final Project Report



McGill
UNIVERSITY

Project Title:

Small Language Models for Classifying
Research Theses and Papers

Project Advisor:

Professor David A. Lowther (david.lowther@mcgill.ca)

Students Involved:

Shivam Aery, 260896005, ECSE 458
Ralph Choucha, 261055349, ECSE 458
Hien Le, 261118519, ECSE 458
Chloe Nasrallah, 260989158, ECSE 458

Company:

The Computational Electromagnetics Research Group at McGill

Abstract

Over the last few decades, the volume of published academic research papers has sharply expanded, going from approximately 1 million papers in 1980, to over 7 million papers by 2014 [1]. This rapid growth has made it increasingly difficult for researchers to efficiently locate relevant work, to stay updated on progress in their fields of expertise, and on identifying unexplored research areas. In light of this growing concern, our project aims to create a proof-of-concept of a Machine Learning system powered by a Small Language Model that can efficiently and locally be run on any McGill researcher's computer. The system allows researchers at McGill to interact with already completed or ongoing research through a graphical user interface that supports question-and-answer style conversations. Given the user's query, the machine retrieves relevant answers from locally stored academic papers. Over the course of the year, our team successfully completed the implementation of a local React-based web interface fully integrated with the backend querying pipeline, along with a scalable and local ingestion pipeline. The backend is capable of processing PDFs, chunking their content, and embedding the chunked content using ChromaDB. We used Javascript, HTML and CSS for the implementation of the frontend. The backend was developed in Python with commonly used machine learning libraries. This POC demonstrates the feasibility of a local research-assistant tool that is designed to assist researchers with their daily tasks.

Table of Contents

Abstract	2
Table of Contents	2
Abbreviations	3
Introduction	4
Background Section	4
Requirements	5
Frontend	5
Backend	5
Design and Results Section	6
Selecting Phi-3	6
Optimizing Phi-3	6
PDF Ingestion	7
High-Level Overview	7
PDF Loading	7
Text Splitting	7
Selecting A Vector DB	8
System Architecture	8
LoRA	8
RAG	9
Prompt Engineering	10
Map-Reduce	10
Prompt Creation	10
Frontend	11
Design	11

Implementation	11
Frontend-Backend Integration	11
Results	12
User Interface	12
Response Output	13
Integration of Tools	13
LangChain	13
Transformers	14
ChromaDB	14
Sentence-Transformers	14
React	14
Axios	14
Modification Of Tools	14
Backend Structure	15
Impact on Society and the Environment	15
Environmental benefits	15
Safety and Risk	15
Use of non-renewable resources	16
Benefits to society	16
Report On Teamwork	16
Shivam	16
Ralph	16
Chloe	17
Hien	17
Conclusion	17
References	19

Abbreviations

SLM - Small Language Model
GUI - Graphical User Interface
POC - Proof of Concept
DB - Database
RAG - Retrieval Augmented Generation
LoRA - Low Rank Adaptation
Q&A - Question and Answering

Introduction

Every year, the number of academic research papers published continues to grow, making it harder for researchers to quickly find the information that they need. This not only leads to slower research progress, but often leads to redundant work and difficulty staying up-to-date with what has already been achieved. This is why we were given the task of creating a tool that simplifies the research process. The application built would be used and maintained by The Computational Electromagnetics Research Group at McGill.

The goal of this project is to build a POC system that helps McGill researchers search, find, and understand different academic papers with more ease. Our system uses a Small Language Model that can run locally on a researcher's computer and interact with them through an intuitive question-and-answer chat interface. In particular, our objective is to implement features that will allow the system to simplify the user's daily research process. Those features include finding papers on a specific topic along with accurate citations, answering questions about a research paper directly through the GUI, summarizing the contents of a paper and helping users discover related work by including articles that discuss the same topic.

Making the process of discovering and understanding already completed research easier would ensure significantly less redundant information in future papers. Future research would become more diverse and would explore more areas of potential interest for researchers. The application could have a significant impact on improving research at McGill and allowing for the daily research process of researchers to be more efficient and effective.

Background Section

Before exploring the entire design process and findings, the readers should familiarize themselves with fundamental concepts in machine learning. We briefly introduced the Small Language Model in the goal of the project. The proper definition of an SLM is that it is a model that is capable of processing, understanding and generating natural language content [2]. Phi-3 is a family of open-source small language models developed by Microsoft [3]. These models come in various sizes such as Phi-3-mini-4k and Phi-3-medium-4k [3]. When using a model to interact with input data, we need a means of storing the input data in a way that is understandable to the SLM. This is where we need to use embeddings which are numerical representations of the inputs that capture the semantics and positions of tokens in the input [2]. We then need to store these embeddings in a vector database. In our case we have chosen ChromaDB, which is an open-source vector store which means that it can be utilized as a database that can store and retrieve embeddings [4]. This vector database provides the functionality of similarity searching which is useful when comparing the input data to the user's query [5]. In order to have our model provide context accurate answers, two main output optimization pipelines have been considered: RAG and LoRA. The implementation we ended up choosing after exploring them is Retrieval-Augmented Generation (RAG) [6]. The concept of RAG relies on searching the top k similar text chunks to the user's query, and passing them with the prompt for answer generation [6]. This helps the SLM generate more relevant responses to the user query given the added context directly sourced from the papers. The other method we experimented with as mentioned earlier is LoRA, or Low-Rank Adaptation. LoRA is a method of training a model that is

lightweight and doesn't require retraining the entire model from scratch [16]. It involves adding an additional layer on top of the original model and fine tuning the answer generation through that layer.

Requirements

The main problem that we are addressing in this project is the increasing difficulty that researchers face as the volume of published research papers increases. Although there are search engines and libraries that give access to millions of uploaded papers, sorting through them and comparing them requires significant manual effort. It becomes an extremely daunting task for researchers to figure out which topics have already been looked into and compare similar studies. The Computational Electromagnetics Research Group at McGill University faces that same challenge, often spending significant time locating articles related to their work and ensuring that they are not unintentionally duplicating existing research.

That is why our project focuses on creating a POC of a system that allows researchers to interact with different papers more efficiently through a Small Language Model that runs entirely on their local machine. The system must meet a set of requirements described below.

The project should run completely locally to address data privacy concerns and to have complete control over the application's components. The end result must be able to run on a standard laptop, which we defined as a laptop that has been made within the last 5 years and contains a dedicated GPU at a minimum. To run the application faster and more seamlessly, a laptop with a GPU is necessary in order to avoid the slower CPU processing. The SLM that we are selecting should be as lightweight as possible, striking a balance between size and output detail and correctness, meaning that it should have the smallest number of parameters possible while still maintaining enough accuracy to produce domain specific answers. The application should also be designed to scale up such that it can accept large volumes of new research paper material as input to the model.

Frontend

For the frontend, the user should be able to interact with a simple and intuitive local GUI. It should provide the interface for the question and answer chat system, where users can enter their queries and receive context-aware answers from the backend pipeline. The frontend should implement the chat page in a clean manner, replacing a command line interface implementation. This is to address the user experience requirement of the project so that we can provide the users with a cleaner way of querying the model. The frontend should also implement some McGill styling elements, since it is done for one of the research groups at the university. Lastly the UI should be designed so that it can handle future additions to the application, and thus be freely expanded. This is where component based frameworks such as React can help achieve that.

Backend

The backend must use a free and open-source pre-trained small language model for response generation. It must implement a pipeline that will properly process the research paper PDFs. The implementation of the backend should be done in a way that optimizes the model's performance such that it can run as fast as possible on more limited hardware, while improving the model's response accuracy. The model's pretraining contains a lot of useful data already. However, questions that the

users may have will include concepts and specific items contained within the research papers. Therefore, the model should be optimized to take those into account. It should also provide a scalable way of dealing with a large amount of research content (by large we mean somewhere in the hundreds of research PDFs).

Design and Results Section

Selecting Phi-3

To build our POC, we needed to select an open-source model that was lightweight, computationally efficient and that demonstrated reliable performance in comparison to other publicly available SLMs. We selected Phi-3 which is a family of various models of different sizes. Phi-3 is open-source and despite being smaller than other popular models such as Gemma, Llama and Mistral, Phi-3 performs exceptionally well for its size factor. In Figure 1 it can be observed that Phi-3.5-mini performs similarly to many other models that have roughly twice its number of parameters. Moreover, it can be seen that there are variations of Phi-3 such as Phi-3.5-MoE that perform even better than many other popular models despite having a much smaller size. We began our experiments with the phi-3-mini-4k-instruct model since it has the smallest size [8]. We needed to work with the smallest model possible in order to ensure that the highest number of users can reliably run our model locally and efficiently. Beginning our experiments with the smallest model meant that we could scale the size of the model as needed as we ran our experiments, ensuring that we do not use a larger model size than necessary for creating our POC.

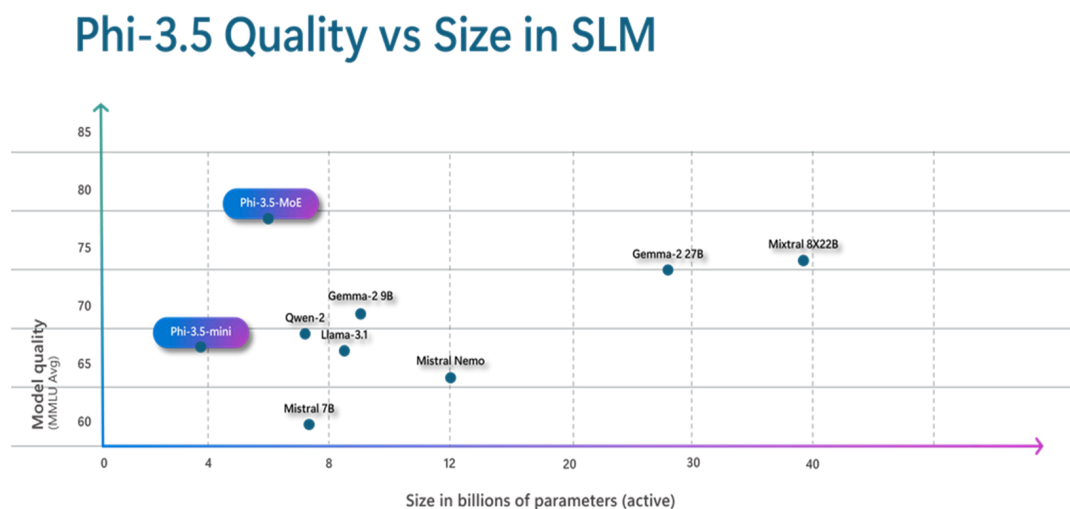


Figure 1. Phi-3 Performance Compared To Other Models [9]

Optimizing Phi-3

As we began our initial experiments with Phi-3, we quickly noticed that the model ran inefficiently on local hardware. In order to mitigate this issue, we implemented two performance fixes. The first performance enhancement was using CUDA to perform GPU acceleration. This greatly improved the time it took to load the model and to run the model when testing various prompts. The second performance improvement involved adding 4-bit quantization. Quantization is a technique that

reduces the computational and memory costs of running inference on SLMs [10]. Quantization greatly decreases the overall time it takes to get an answer from the model and the amount of memory the model takes [10]. After applying 4-bit quantization and GPU acceleration, the model ran in reasonable runtimes of about 20-30 seconds. Our decision to apply 4-bit quantization, which is the most aggressive form of quantization, was necessary because we wanted to ensure that the model took as little space in memory as possible and that the model ran as quickly as possible. However, despite our efforts in optimization, due to phi-3-mini-4k-instruct having a very small context length, we needed to switch to Phi-3-medium-4k-instruct in order to properly process the research paper PDFs given to us by the project advisor.

PDF Ingestion

High-Level Overview

The first feature our application needed to implement was PDF ingestion. Given that there were multiple large technical PDF research papers, we needed to implement a means of parsing and storing all these PDFs locally. Our ingestion pipeline serves three main purposes: splitting the input PDF papers and obtaining chunks, storing the chunks in a local vector database called ChromaDB and dynamically searching for all contextually relevant chunks in the vector database and it must be able to detect and add any new PDF inputs to the database. Splitting the PDF papers involved dividing the documents into smaller segments that preserve context [11]. These segments are called chunks and these are then stored into a local instance of ChromaDB. However, the chunks are not stored directly in ChromaDB. Instead, these chunks are passed through an embedding model which converts the chunks into embeddings. The main design decisions in our PDF ingestion pipeline involve the following: Choosing a text splitter, choosing a vector DB and choosing a PDF loader.

PDF Loading

The PDF loader we selected for loading all of the PDFs was UnstructuredPDFLoader which is a loader in the LangChain module [12]. It is mainly designed to extract clean, structured text from PDFs [12]. We selected this loader because it not only extracts the content but it also organizes text by identifying important semantic elements such as Titles, Headings, Paragraphs, Tables, Lists and page breaks [12]. This feature is a major benefit to our application as we want to identify these different elements of the research papers so that it is easier for Phi-3 to understand the research paper PDF content.

Text Splitting

There are various methods of splitting text but the one that we selected is called RecursiveCharacterTextSplitting. The method consists of an algorithm that takes as input the target length for each chunk produced and a list of separators (ex: newline, period, comma) which are breaking points for each chunk [13]. The algorithm recursively splits the entire text at every occurrence of a separator in the list of separators specified [13]. Our reasoning for choosing this method of splitting the text in the research paper PDFs is that it is flexible since it is a type of splitting that splits text based on a separator and these separators can be modified [13]. This results in it being able to handle both sentences and paragraphs [13]. It is also a very efficient method since it splits at

natural breakpoints such as newlines, ends of sentences or ends of paragraphs which makes it applicable to our use case of splitting technical paper content [13].

Selecting A Vector DB

We needed to store embeddings locally but in a way that is scalable and easy to maintain. Due to this requirement, we decided to use a vector database. A vector database is a database that stores vector embeddings for efficient retrieval [14]. Our methodology for selecting a vector DB involves considering multiple factors which were: rapid development, size of our embeddings, local development and suitability for implementing RAG. Since we wanted to build a POC, we prioritized a choice of DB that would allow for the fastest development. Another constraint of our application was that the entire application needed to be local, meaning that no cloud databases could be utilized. As a consequence, we required that the development and testing of the DB would be done locally. Lastly, we needed a DB that would integrate well into a RAG pipeline since the RAG architecture was the one we chose for retrieving our documents from the vector DB. ChromaDB satisfied all of these constraints and was simple to use along with clear documentation [15]. Therefore, we used ChromaDB as our choice of DB.

Prior to utilizing ChromaDB, we did attempt to store all the embeddings of our chunks locally in a JSON format. However, we quickly understood that ChromaDB was needed since storing all embeddings in a local JSON is not scalable. In fact, storing all of the embeddings as a local JSON would have made retrieval of contextually relevant documents difficult which only reinforced our decision to use ChromaDB.

System Architecture

LoRA

During the project's timeline we attempted two separate implementations to fine-tune our small language model. We started by implementing the RAG pipeline but we also worked in parallel on LoRA or Low Rank Adaptation as an alternative. This method essentially consists in freezing the existing parameters of the original model, and on top of that, adding small low-rank matrices that adjust the output of the model to obtain more context specific answers [7]. The number of parameters can amount to billions even in the case of a small model such as Microsoft's Phi-3 mini which has 3.8 billion parameters [8]).

In theory, if implemented correctly, LoRA would save us the complex cost of retraining the model from scratch, which is why we considered it in the first place and wanted to keep it as an alternative. However, we quickly found out that given our project's constraints, and the input data being the raw text from PDFs, we simply couldn't efficiently apply LoRA. When passing the document chunks as tokens to the training loop, the model only learns the formatting, the model does not learn the context or data that is contained within the research papers. This is because the training data needs to be in the form of a set of questions and answers that the model can base itself off of. Keeping only the text chunks as our training dataset will not get the model to answer our users' potential queries.

To make LoRA work, we would therefore have to build a large enough dataset of questions and answers that covers the contents of the entire research paper library, which would require the use of external tools and additional review of the dataset by domain experts. This renders the implementation

complex to manage and unrealistic for our project. Furthermore, this solution would have scalability concerns: for every new research paper added to the existing collection, we would have to expand the training dataset on top of re-running the LoRA training loop to keep the model's knowledge base up to date. Thus we quickly rejected LoRA as a possible solution and focused solely on improving our RAG pipeline.

RAG

RAG was our choice of system architecture over LoRa. RAG is an architecture that enhances the output of a SLM so that it references authoritative and contextually relevant knowledge outside of its pre-trained training data before generating a response to a user's query [16]. This strengthened our applications ability to send domain specific knowledge to Phi-3 so that Phi-3 can offer responses that are contextually aware without having to train Phi-3. The major advantage of not needing to train RAG is that training a pre-built model such as Phi-3 on a new dataset can take a significant amount of effort and time. In fact, given the hardware we were utilizing which consisted of our personal computers with GPUs, it was unrealistic to train Phi-3 on all of the research paper PDFs provided to us by the project advisor. RAG solves this problem by allowing us to use the pre-built Phi-3 without needing to modify the model at all. This significant advantage of RAG made it the optimal solution to creating our POC.

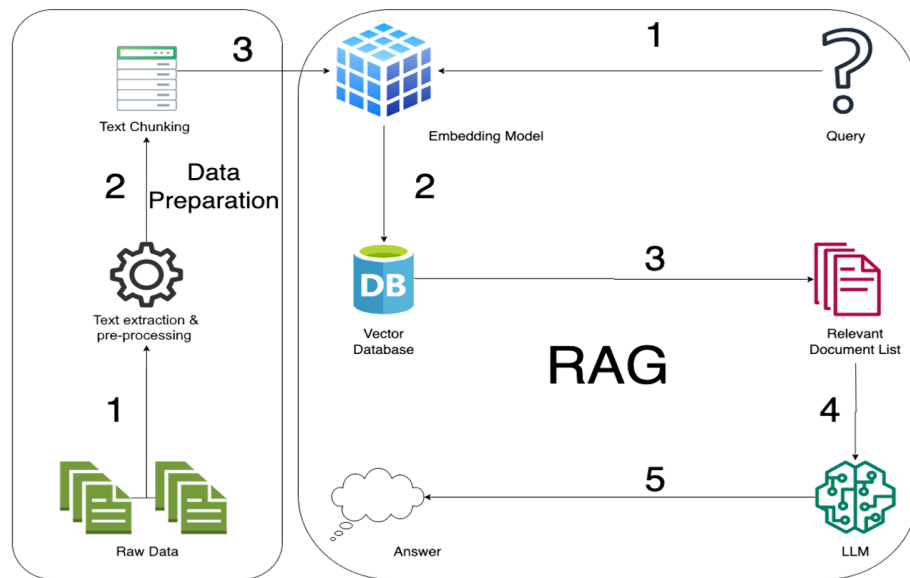


Figure 2. RAG Architecture [17]

Figure 2 describes RAG at a high-level. The first step is to, as mentioned before, perform PDF pre-processing and to extract the text of the input data into chunks. Once these chunks are created, they need to be passed through an embedding model. In our case, we used a pre-built embedding model from the sentence-transformers library. Afterwards, the embedded chunks are stored into our local instance of ChromaDB. A user in the frontend will enter a user query and ChromaDB will perform a similarity search [18]. ChromaDB will obtain the most relevant chunks to the user query which results in building a combined context of the most relevant passages in the input PDFs. The context that consists of all the combined relevant chunks are then passed to Phi-3. Phi-3 will then generate a response by using those relevant chunks as context in order to produce an intelligent response. This response will then be sent from the backend to the frontend and displayed to the user.

Prompt Engineering

Map-Reduce

Instead of passing our chunks directly to the model, we use a method called map-reduce to send our chunks to Phi-3. The challenge that comes with passing chunks to phi-3-mini-instruct is that the model's context length is small which does not allow for a large amount of content to be passed to the model. Initially when attempting to directly pass all of our chunks directly to the model, we noticed that the model could not handle all of the context since the context was too large. As a result, many of our model outputs were not contextually relevant and often contained unintelligible outputs. To solve this problem, we decided to implement the Map-Reduce text-summarization method [19]. The Map-Reduce method was chosen because it is especially effective for large documents as inputs [19]. Map-Reduce works by taking the chunks and summarizing them independently which is the "Map" step [19]. Afterwards, the summaries of each chunk are combined into a final summary which is sent to the model [19]. These chunk summaries and the final summary are all generated by the model [19] by using prompts. This method severely reduced issues with context-length and allowed for our model to output answers that are contextually relevant and correctly cite sources.

Prompt Creation

```
# --- Map Prompt ---
map_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a technical research assistant. "
     "You must use only the information in the provided context to answer the question. "
     "You cannot use outside knowledge, assumptions, or hallucinate facts. "
     "If the context does not provide any relevant information, respond exactly: NO RELEVANT INFO."),
    ("user",
     "Context:\n{doc_text}\n\nQuestion:\n{question}\n\n"
     "Instructions:\n"
     "1. If the context directly answers the question, summarize the answer clearly (max 100 tokens).\n"
     "2. If the context does not answer the question but provides background or related information, "
     "summarize how the context relates to the question (max 150 tokens).\n"
     "3. If the context is unrelated, respond exactly: NO RELEVANT INFO.\n\nAssistant:\nSummary:")
])

# --- Reduce Prompt ---
reduce_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a precise technical summarizer. Use only the provided summaries."),
    ("user",
     "Partial answers (each with its source):\n{partial_answers}\n\n"
     "Question:\n{question}\n\n"
     "Instructions:\n"
     "1. Discard any entries that are 'NO RELEVANT INFO.'\n"
     "2. Choose the most relevant and complete answer that directly addresses the question.\n"
     "3. If multiple are similar, merge minimally while preserving factual clarity.\n"
     "4. If none directly answer, respond exactly: NO RELEVANT INFO.\n\n"
     "Final Answer:")
])
```

Figure 3. Creating The Prompt For Map-Reduce

Figure 3 displays the two prompts we used for the Map step and the Reduce step of the Map-Reduce method. The method that we used to design both of the prompts included brute-force testing of different prompts and determining whether or not they provided an adequate answer. We focused on first providing a persona for the model which allows the model to more accurately understand its purpose in our application [20]. We specified that the model is a technical research assistant making sure that it understands that our research paper PDFs are highly technical content which helps the model to provide more technically relevant answers. We then specified the constraints of its response to ensure that the model did not rely on any external knowledge. We imposed this requirement because we noticed throughout testing that the model would attempt to conduct external searches

despite the query answers being in the provided context. We explicitly told the model to not use any external knowledge and to only rely on the provided context that is given through RAG.

We provided the model with clear numbered instructions on how to respond and we instructed it to respond with “NO RELEVANT INFO” if it could not answer a user’s query. We found throughout testing that the model did clearly indicate if it had no relevant answer in the context to answer a user’s query and we discovered that this combination of prompts, along with the map-reduce method allowed for the model to provide intelligent and contextually relevant answers to user queries.

Frontend

Design

The frontend was implemented using the React framework for its simplicity and scalability. For the purposes of this project, we have decided to keep the frontend simple while maintaining a degree of professional styling. This helps satisfy the user experience requirements of our project but also provides a clean way for the application’s users to pass their queries.

The design work began through the creation of a figma mockup that illustrated the ideal appearance of the final user interface. It provided a clean example of a chat page that mimicked conventional AI chatbot applications. This allowed us to gather early feedback from our supervisor and to validate the expected functionality of the end result, ensuring that the UI aligns with his expectations. We have also found an existing online service whose functions were similar to our project (scite.ai) and we presented it to the supervisor to further guide functionality and design. Since the application is being built for the Computational Electromagnetics Research Group at McGill, we used their website color scheme as inspiration for the design of our application.

Implementation

We began by creating the react application and adding initial elements. We implemented a very simple version of the chat page that only contained the chat logic and display. In order to test its functionality, we echoed the user input as a placeholder answer. This placeholder answer was where the model’s output would be generated and displayed. We then continued building the UI by adding the necessary styling elements for the chat page. This included the chat bubbles, the chat frame as well as the input box and submit button. Eventually, a home page was added to augment the UI as well as a navigation bar, a proper color scheme, and small page animations, all in an effort to set-up the frontend for eventual feature expansions as well as improving the user experience.

Frontend-Backend Integration

In order to display the output generated by the model, we have implemented a simple API with the use of the FastAPI python library. In order to handle the frontend’s requests, The API exposes a POST endpoint (/query) that accepts a JSON payload containing the user’s prompt. This payload is validated through a Pydantic request model to ensure proper request format. Once the answer is received, the prompt is passed through to the backend pipeline. The model then generates the answer which is passed back to the API. The answer, which is a python dictionary, contains both the output text from the model and the sources used. These two elements are then taken and passed through a Pydantic

response model in order to serialize those elements into a properly formatted JSON. This payload is then sent back to the frontend for display.

In order to run the API, we have used the Uvicorn web server interface, specifically designed to handle web requests and integrate with FastAPI. It handles the application's POST requests as well as loading the model before running the server, ensuring we don't need to start and stop the model for every query.

Results

User Interface

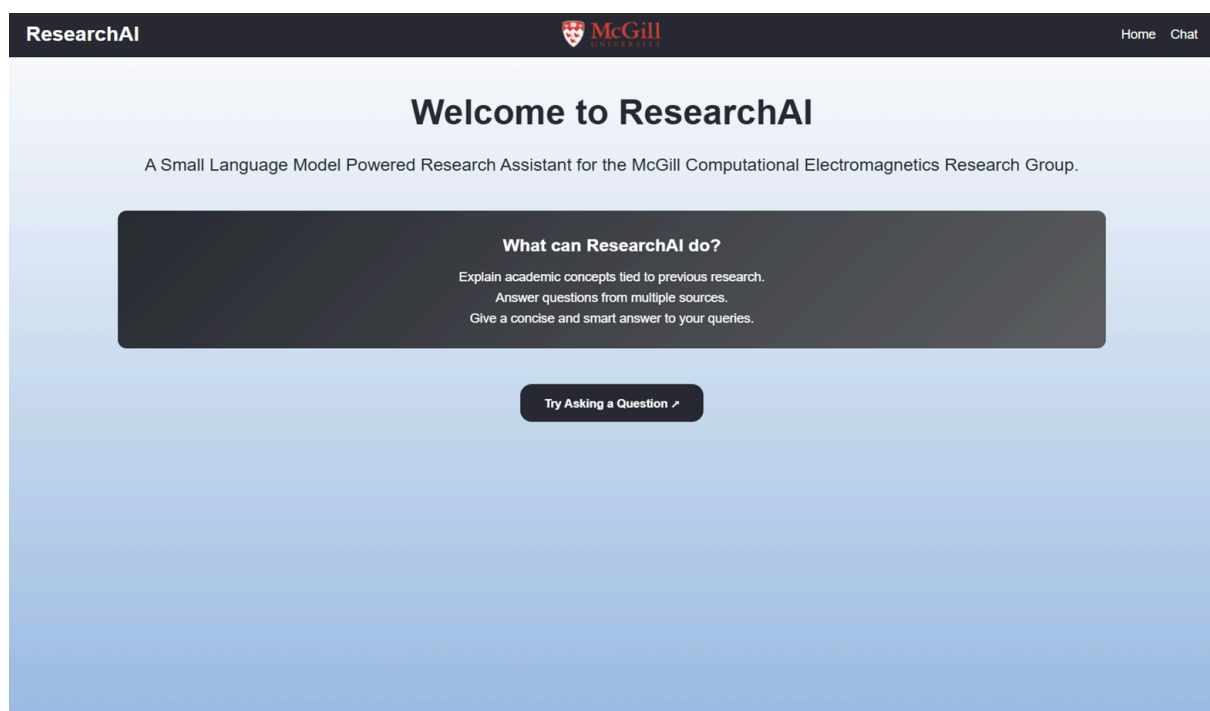


Figure 4. UI Home Screen

Figure 4 demonstrates that our GUI consists of two screens, the first is the home page. The home page gives a brief overview of the capability of the application. We emphasize that the application, ResearchAI is a small language powered research assistant that can be interacted with to serve three main purposes. These three purposes include: explaining academic concepts of previously completed research, answering questions by searching through multiple academic sources and the assistant aims to give a concise and contextually relevant answer to your question. At the top we include the McGill logo, home and chat navigation buttons and at the bottom we include a button to move to the chat screen.

Response Output

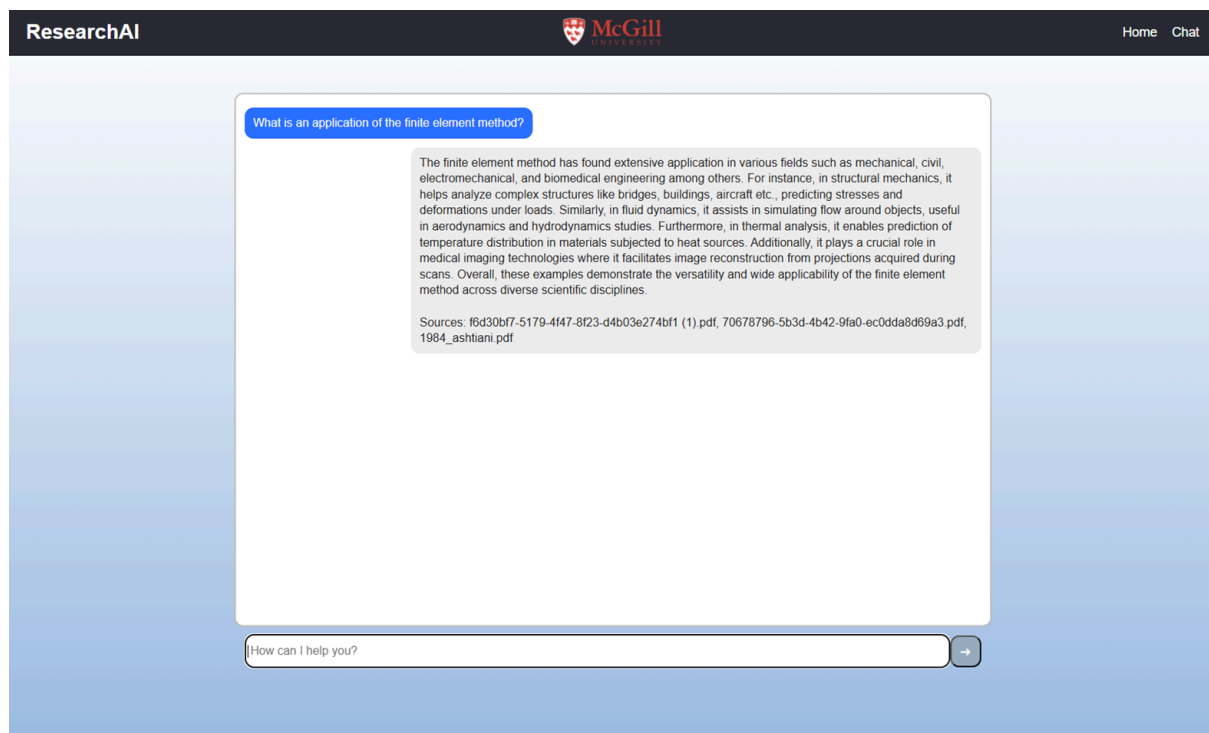


Figure 5. Question and Answer Response Example

We include in Figure 5 an example of the output of the model. The query submitted is “What is an application of the finite element method” which is a question that is based on the contents of the research paper PDFs. It can be observed that the model provided an answer that is contextually relevant and intelligent. The model mentions the fields that the finite element method is used in and gives examples of its usage across various scientific disciplines. Furthermore, the model outputs the sources it used. It can be observed that these sources are a few of the research paper PDFs provided. Therefore, we can conclude that the model is answering questions using information directly from its provided context and that it is not relying on its pretrained knowledge or on external data sources. This confirms that RAG is functioning correctly and that the pipeline of ingesting the PDF papers, configuring the model parameters and creating the pipeline for interacting with Phi-3 has been built correctly.

Given the requirements of the project, we have successfully achieved a POC that proves that it is possible to use a small language model to build an application where a user can interact with and ask questions to the model and get relevant answers to research papers.

Integration of Tools

LangChain

A critical library used throughout our application is LangChain. The main key features of LangChain is that it integrates with ChromaDB, it contains loaders for PDF loading, it can be used to generate a dynamic prompt for the model with a template structure and it can be used to further customize prompts [21]. In our application it was used to obtain the RecursiveCharacterTextSplitter class that is used to split documents and produce chunks. It allowed for the creation of a local instance of

ChromaDB. Moreover, it was utilized to obtain the UnstructuredPDFLoader that was used to implement effective loading of the PDF input data. LangChain was used to import the various functions needed to define chat prompt templates and to implement the map-reduce method. Lastly, LangChain was used for creating a pipeline for loading the model, loading Phi-3 and its relevant configuration settings.

Transformers

The transformers library was used for loading the Phi-3 model. Transformers allowed us to configure different settings of the model such as applying quantization. It was used for mapping the model to the GPU so that GPU acceleration can be used to make loading and using the model significantly more efficient.

ChromaDB

ChromaDB was used for storing the local collection of embeddings of the split documents. It was used for retrieving relevant chunks that are contextually relevant with respect to a user query. This functionality was accomplished via Similarity Search which is a function directly provided by ChromaDB. Similarity search returns the most relevant chunks based on how similar the chunks are to a given user query [18]. The similarity of a chunk is determined based on a metric which can be configured. These metrics include cosine similarity or euclidean distance [18]. In our application, we configured similarity search to get the 3 most contextually relevant chunks given a user query. We selected 3 because we wanted to avoid passing too much context to our model since it has limited context length. We did not modify the distance metric used for Similarity Search and found that euclidean distance worked well.

Sentence-Transformers

The embedding model that we used to create the embeddings of our chunks is called “sentence-transformers/all-mpnet-base-v2” [22]. We selected this embedding model since it is easily accessible and widely used.

React

For our frontend, we chose React as our framework of choice. We chose React because it has excellent documentation with a plethora of resources and tutorials online [23]. React code uses a Component-Based architecture which makes code built using React easier to maintain and reuse [23].

Axios

In order to retrieve the responses generated from the frontend and to display them in the frontend, we used the Axios library. The axios library was used to define a POST endpoint in order to retrieve responses generated from using FastAPI in the backend.

Modification Of Tools

We did not modify any of the tools integrated with our project.

Backend Structure

In order to produce effective results, it was imperative for us to create a production level backend structure. We made the design decision of emphasizing a modular file structure where each file would serve a distinct purpose. Our file structure consisted of 6 main files: `api.py`, `cli.py`, `config.py`, `ingest.py`, `model.py` and `qa_service.py`. `Api.py` has the purpose of integrating the backend with the frontend via FastAPI. `Cli.py` had two main use cases: obtaining easily readable model outputs with their corresponding queries and providing an interactive CLI for interacting with the model. We implemented this CLI for testing purposes prior to building our frontend. The CLI allows for specifying the application to perform ingestion of all local PDF research files or for specifying a user query to send to the model.

`Config.py` served the purpose of specifying environment variables to specify the parameters of the model, the chunk size and the model name. `Ingest.py` was used for the ingestion of the PDF input documents and relevant pre-processing. `Model.py` is the file in which we imported the model and performed model optimizations such as quantization. Lastly, `qa_service` is where we defined the prompts for the model and where we used the map-reduce method to perform Q&A with the model.

Impact on Society and the Environment

While our goal in this project was to create a proof-of-concept of a local web application that helps sort through research papers, the resulting application has broader implications that go beyond completing certain requirements, and that may affect both society and the environment.

Environmental benefits

Our initial environmental benefit comes from using a Small Language Model as opposed to a LLM. Lightweight SLMs have been shown to consume significantly less energy and water than their LLM counterparts, with some studies showing reductions of up to 70% [24]. In addition to using a smaller model, another significant impact comes from the local use of our application as opposed to computing on the cloud. Cloud-based systems require data centers that consume large amounts of electricity, and can consume much more power to perform the same tasks that can be performed on a low-power local device [25].

Safety and Risk

Despite these benefits, our system also introduces certain risks, mainly related to the underlying model. SLMs have a possibility to inherit certain biases from the datasets by which they were trained. Studies have shown that LLMs may pose challenges due to their susceptibility to biases, which can lead to skewed results and potentially exacerbate historical and societal biases [26]. This poses a risk in our application, as the model may generate inaccurate results, or biased interpretations of researched topics. Additionally, because the model may perform better on certain subjects than on others, users may encounter inconsistencies in the accuracy of the responses they receive.

Use of non-renewable resources

As the designer and manufacturer of the software, we mostly consume renewable electrical resources. However, taking into consideration all stages across the entire project, there are many uses of physical components that can be classified as non-renewable resources. Our laptops that run the SLM contain semiconductor chips, memory and CPUs that were extracted through mining. These physical components can be hard to recycle. The plastic covers of the laptops and metal components such as aluminum and copper are also accounted for during the manufacturing process. During the distribution and use by consumers, our software doesn't require physical transportation, and we only consume local hardware resources. These factors can lead to degradation in hardware and the need for hardware replacement. The software doesn't require physical disposal and thus would have no environmental impact but the hardware that runs the product needs to be disposed of as electronic waste and can be difficult to recycle.

Benefits to society

This project allows researchers to more efficiently understand completed research work and to obtain relevant insights from completed work so that they can more effectively complete their daily research tasks. The project will allow researchers to complete research quicker and more accurately allowing for an increase in the quality of research completed. Financially and logistically there can be a reallocation of resources to concentrate on completing novel research rather than potentially duplicating past research. As a result, research diversity will increase and be of better quality.

Report On Teamwork

Shivam

He completed the ML research required to obtain the necessary background information needed to complete the project. Shivam completed the ingestion pipeline of the PDF which covers the PDF ingestion, PDF pre-processing and the storage of the PDFs as chunks. Shivam implemented the local instance of ChromaDB. Moreover, he worked on creating the map-reduce Q&A chain used to interact with Phi-3. He designed the prompts used for the map-reduce steps and participated in the testing of the Phi-3 model. He researched and implemented RAG. He coordinated group meetings and the project timeline.

Ralph

Ralph found the method used to integrate Phi-3 into the backend. He performed initial tests of the model's capabilities and found the method used to perform user queries and receive relevant responses. Moreover, he performed performance tests of the model along with Shivam in order to verify the model's memory usage and efficiency on local hardware. Ralph had found an initial means of storing the PDF embedding data which consisted of storing the data as JSON. He researched LoRa and implemented a simple training loop for LoRa. He evaluated the output of LoRa and found LoRa was not the best choice of architecture for our use case. He integrated the frontend with backend components via FastAPI. He assisted in stylizing the frontend. He worked with Shivam in testing the output of the Phi-3 model in the backend and finding potential sources of improvement to the model's output.

Chloe

Chloe was tasked with working solely on the front-end of the project. She found a website that was used as inspiration for the functionality of our application. She worked on the Figma mock-up design on the UI. Moreover, she created the React project and the UI interface of the chat page. She worked with Ralph on designing and creating the home page of the application. She created the various components of the frontend such as the chat box and other critical components.

Hien

Hien is another crucial member for the backend team, focusing on the testing and researching phase of the model. Hien tested the original Phi-3 model and tried running it on Intel I7 GPU with Intel Iris graphics. He concluded an overuse in memory and power resources. He also did preliminary research on ChromaDB as the local host for the pdf database. Hien did thorough research on Low Rank Adaptation. After the break, since he finds it challenging to work more on the backend, he decides to focus more on the front end work, was able to explore some concepts regarding some front end framework, learning the Typescript text bubbles for user prompts. He also takes care of some administrative work, taking care of the printing of the design day poster and communicating with professors.

Conclusion

Throughout this semester, our group progressed from having limited familiarity with AI and machine learning to successfully developing a functional and well-structured RAG implementation. We implemented a fully functional backend and frontend that allows for users to successfully query the research paper PDFs provided to us by the project advisor. The system is fully local and has met the core requirements of the project. We have achieved a UI that is user-friendly and fully integrated with the backend. We have completed a POC that can be built upon in a way that is scalable and effective for any future endeavors.

The next steps of the project should include improving the model output through further tuning of the parameters, adding additional UI features to enhance the usability of the application, conducting user acceptance testing with the Project Advisor in order to improve the model output and incorporating cloud storage for ensuring that the research paper PDFs would be stored in a scalable manner. Other improvements that would be necessary would include using a client-server architecture to ensure that the model is running using cloud based resources for answering queries rather than local resources. An impactful future effort would be to implement fully conversational AI. Currently the model understands natural language queries and responds adequately but the model does not have a past memory of the context of the conversation.

Throughout the project, we all have improved in project management and communication within a team. We have practiced following a proper design process and followed the steps of implementing a POC from its inception phase till its completion. We have strengthened our technical skills by researching and implementing commonly utilized machine learning methods and the RAG architecture. We have developed our ability to create user-friendly interfaces and their custom styling.

References

- [1] M. Fire and C. Guestrin, “Over-optimization of academic publishing metrics: observing Goodhart’s Law in action,” *GigaScience*, vol. 8, no. 6, May 2019, doi: <https://doi.org/10.1093/gigascience/giz053> (accessed Dec 7, 2025).
- [2] R. Caballar, “What are small language models?” *ibm.com*. <https://www.ibm.com/think/topics/small-language-models> (accessed Dec 7, 2025).
- [3] M. Bilenko, “Introducing Phi-3: Redefining what’s possible with SLMs” *azure.microsoft.com*. <https://azure.microsoft.com/en-us/blog/introducing-phi-3-redefining-whats-possible-with-slms/> (accessed Dec 7, 2025).
- [4] A. Awan, “Chroma DB Tutorial: A Step-By-Step Guide” *datacamp.com*. <https://www.datacamp.com/tutorial/chromadb-tutorial-step-by-step-guide> (accessed Dec 7, 2025).
- [5] R. Cohen, “What is a Vector Database & How Does it Work? Use Cases + Examples” *pinecone.io*. <https://www.pinecone.io/learn/vector-database/> (accessed Dec 7, 2025).
- [6] S. Aquino, “What is RAG: Understanding Retrieval-Augmented Generation” *qdrant.tech*. <https://qdrant.tech/articles/what-is-rag-in-ai/#> (accessed Dec 7, 2025).
- [7] J. Noble, “What is Lora (low-rank adaption)?,” *IBM*, <https://www.ibm.com/think/topics/lora> (accessed Dec. 4, 2025).
- [8] “microsoft/Phi-3-mini-4k-instruct · Hugging Face,” *Huggingface.co*, Jul. 11, 2024. <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct> (accessed Dec 7, 2025).
- [9] AdinaTru, “Discover the New Multi-Lingual, High-Quality Phi-3.5 SLMs,” *TECHCOMMUNITY.MICROSOFT.COM*, Aug. 22, 2024. <https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/discover-the-new-multi-lingual-high-quality-phi-3-5-slms/4225280> (accessed Dec. 6, 2025).
- [10] “Quantization,” *huggingface.co*. https://huggingface.co/docs/optimum/en/concept_guides/quantization (accessed Dec. 6, 2025).
- [11] Arooj, “Why Text Splitting Matters in Modern RAG Systems,” *Chitika: Explore Retrieval Augmented Generation Trends*, Mar. 16, 2025. <https://www.chitika.com/importance-text-splitting-rag/> (accessed Dec. 6, 2025).
- [12] Sangitapokhrel, “Different Methods to Read PDF Files in LangChain,” *Medium*, Nov. 16, 2025. <https://medium.com/%40sangitapokhrel911/different-methods-to-read-pdf-files-in-langchain-5d547206bcef> (accessed Dec. 6, 2025).
- [13] Y. Siddique, “Understanding Recursive Character Text Splitting,” *Medium*, Jun. 05, 2024. https://medium.com/@yasir_siddique/understanding-recursive-character-text-splitting-8419518db6f4 (accessed Dec. 6, 2025).

- [14] R. Schwaber-Cohen, "What is a Vector Database & How Does it Work? Use Cases + Examples," *pinecone.io*, May 03, 2023.
https://www.pinecone.io/learn/vector-database/?utm_term=vector%20database&utm_campaign=vector-db-us&utm_source=adwords&utm_medium=ppc&hsa_acc=3111363649&hsa_cam=16569728076&hsa_grp=135276647900&hsa_ad=587750423880&hsa_src=g&hsa_tgt=kwd-5220071349&hsa_kw=vector%20database&hsa_mt=e&hsa_net=adwords&hsa_ver=3&gad_source=1&gad_campaignid=16569728076&gbraid=0AAAAABrtGFDsiT9gZ84sLHV8nYE-MFj0Y&gclid=CjwKCAiAxc_JBhA2EiwAFVs7XImqzd7HaxNhCOAPSD_PwCdZHZvKZUMtl0mqNYMdko9KojrW18TyKRocNMwQAvD_BwE (accessed Dec. 6, 2025).
- [15] Priyansh Khodiyar, "RAG Vector Database Selection For Developers," *CustomGPT*, Oct. 08, 2025. <https://customgpt.ai/rag-vector-database-selection/> (accessed Dec. 6, 2025).
- [16] Amazon, "What is RAG? - Retrieval-Augmented Generation Explained - AWS," *Amazon Web Services, Inc.*, 2024. <https://aws.amazon.com/what-is/retrieval-augmented-generation/> (accessed Dec 7, 2025).
- [17] Uğur Özker, "Advanced RAG Architecture," *Software Diagrams*, Jul. 20, 2025.
<https://softwarediagrams.com/diagrams/advanced-rag-architecture/> (accessed Dec. 6, 2025).
- [18] GeeksforGeeks, "Introduction to ChromaDB," *GeeksforGeeks*, Jul. 2025.
<https://www.geeksforgeeks.org/nlp/introduction-to-chromadb/> (accessed Dec 7, 2025).
- [19] Damilare Keshinro, "Baeldung," *Baeldung on Computer Science*, Apr. 15, 2025.
<https://www.baeldung.com/cs/langchain-text-summarize> (accessed Dec. 6, 2025).
- [20] Pieces, "10 Prompt Engineering Best Practices - Pieces - Medium," *Medium*, Oct. 07, 2024.
<https://pieces.medium.com/10-prompt-engineering-best-practices-a166fe2f101b> (accessed Dec 7, 2025).
- [21] Idan Novogroder, "RAG Pipeline: Example, Tools & How to Build It," *lakeFS*, Jul. 15, 2024.
<https://lakefs.io/blog/what-is-rag-pipeline/> (accessed Dec 7, 2025).
- [22] "sentence-transformers/all-mpnet-base-v2 · Hugging Face," *huggingface.co*.
<https://huggingface.co/sentence-transformers/all-mpnet-base-v2> (accessed Dec 7, 2025).
- [23] Ashutosh Gaikwad, "Why React is a Great Choice for Your First Front-End Framework," *Medium*, Oct. 29, 2024.
<https://medium.com/@gaikwadashutosh35/why-react-is-a-great-choice-for-your-first-front-end-framework-805c6c9943bd> (accessed Dec. 6, 2025).
- [24] A. Sundaram, S. Kamdar, and S. Kumar, "AI's Environmental Cost: Comparing Resource Consumption Between SLMs and LLMs Across Queries," in *Proc. 5th Int. Conf. on AI Research (ICAIR 2025)*, 2025. doi: 10.34190/icaire.5.1.4345 (accessed Dec 7, 2025)
- [25] A. Vishwanath, F. Jalali, K. Hinton, T. Alpcan, R. W. A. Ayre, and R. S. Tucker, "Energy consumption comparison of interactive cloud-based and local applications," **IEEE J. Sel. Areas*

Commun.*, vol. 33, no. 4, pp. 616–626, 2015, doi: 10.1109/JSAC.2015.2393431 (accessed Dec. 6, 2025).

[26] Y. Guo et al., “Bias in large language models: Origin, Evaluation, and mitigation,” arXiv.org, <https://arxiv.org/abs/2411.10915v1> (accessed Dec. 7, 2025).